Experimental Analysis of Strip Packing Algorithms

Comparative Study of Exact, Constructive, and Shelf Heuristics
with Machine Learning Algorithm Selection

Andrei Mărisac

Facultatea de Informatică
Universitatea Alexandru-Ioan Cuza Iași

May 2026

Contents

Abstract

We present an experimental analysis of seven algorithm families for the Two-Dimensional Strip Packing Problem (2SP): the Bottom-Left-Fill (BLF) constructive heuristic with four sorting strategies, the Next Fit Decreasing Height (NFDH) and First Fit Decreasing Height (FFDH) shelf heuristics, the Skyline heuristic (four sorting strategies), a Simulated Annealing (SA) metaheuristic, and an exact method based on Google OR-Tools CP-SAT. Experiments on

60 benchmark instances (Bengtsson 1982; Berkey & Wang 1987; Martello & Vigo 1998) show that BLF achieves a mean gap of 8.1% above the area lower bound, while the Skyline heuristic produces significantly larger gaps (mean 21.6%) due to its inability to fill concave cavities below tall items. SA, using Skyline as an inner evaluator, achieves a mean gap of 9.9% on random instances with a 5 s budget, beating BLF best-of-four on 14 of 16 tested instances. We train two ML algorithm selectors on 17 geometric features: a Random Forest (RF) achieving 45% exact accuracy and 0.75% mean regret on a 100-instance held-out test set, and a PyTorch MLP achieving 34% accuracy with 0.94% mean regret. Even on misclassified instances both models incur sub-1% height regret, confirming the practical value of data-driven algorithm selection.

1 Introduction

The *Two-Dimensional Strip Packing Problem* (2SP) asks: given a strip of fixed width W and an unbounded height, and a set of n axis-aligned rectangles $\{(w_i, h_i)\}_{i=1}^{n}$, pack all rectangles into the strip without overlap and minimise the resulting height H. The problem is NP-hard in the strong sense [7].

Practical applications include paper and sheet metal cutting, VLSI floorplanning, advertisement placement, and container loading. This report evaluates the algorithms listed in Table 1.

Table 1: Algorithms studied in this report.

ID	**Algorithm**	**Class**	**Complexity**
BLF-DH	BLF, Decreasing Height	Constructive heuristic	$O(n^3)$
BLF-DW	BLF, Decreasing Width	Constructive heuristic	$O(n^3)$
BLF-DA	BLF, Decreasing Area	Constructive heuristic	$O(n^3)$
BLF-DP	BLF, Decreasing Perimeter	Constructive heuristic	$O(n^3)$
NFDH	Next Fit Decreasing Height	Shelf heuristic	$O(n \log n)$
FFDH	First Fit Decreasing Height	Shelf heuristic	$O(n^2)$
Sky-DH	Skyline, Decreasing Height	Skyline heuristic	$O(n^2)$
Sky-DW	Skyline, Decreasing Width	Skyline heuristic	$O(n^2)$
Sky-DA	Skyline, Decreasing Area	Skyline heuristic	$O(n^2)$
Sky-DP	Skyline, Decreasing Perimeter	Skyline heuristic	$O(n^2)$
SA	Simulated Annealing (Skyline inner)	Metaheuristic	$O(n \cdot iter)$
CP-SAT	OR-Tools CP-SAT	Exact (60 s limit)	Exponential
ML-RF	Random Forest Selector	Algorithm selection	$O(1)$ inference
ML-MLP	Neural Network (MLP)	Algorithm selection	$O(1)$ inference

2 Problem Formulation

Let $\mathcal{R} = \{(w_i, h_i)\}_{i=1}^{n}$ be the set of rectangles and $W \in \mathbb{Z}^+$ the strip width. The placement of rectangle i is determined by its bottom-left corner (x_i, y_i). The optimisation problem is:

$$\begin{aligned} \text{minimise} \quad & H && (1)\\ \text{subject to} \quad & x_i + w_i \le W, \quad y_i + h_i \le H, \quad x_i, y_i \ge 0 \quad \forall i && (2)\\ & \text{no overlap between any two rectangles.} && (3) \end{aligned}$$

A useful lower bound is the *area lower bound*:

$$\mathrm{LB}_{\text{area}} = \left\lceil \frac{\sum_{i=1}^{n} w_i h_i}{W} \right\rceil. \tag{4}$$

The *gap* of a solution with height H relative to this bound is:

$$\text{Gap} = \frac{H - \text{LB}_{\text{area}}}{\text{LB}_{\text{area}}} \times 100\%. \tag{5}$$

3 Algorithms

3.1 Bottom-Left-Fill (BLF)

BLF is a greedy constructive heuristic introduced by Baker et al. [1] and formalised by Chazelle [5]. After sorting the rectangles by a chosen key, each rectangle is placed at the *lowest* feasible y-coordinate, then as far *left* as possible at that level. Candidate y-levels are $\{0\} \cup \{y_j + h_j : j \text{ placed}\}$, i.e. the bottom of the strip or the top edge of any placed rectangle. Similarly, candidate x-positions are $\{0\} \cup \{x_j + w_j\}$.

Four item orderings are compared: **DH** (decreasing height), **DW** (decreasing width), **DA** (decreasing area), and **DP** (decreasing perimeter).

3.2 Shelf Heuristics (NFDH, FFDH)

Shelf heuristics [6] pack items in horizontal *shelves*. Items are sorted by decreasing height.

NFDH: keeps one open shelf; when the next item does not fit horizontally, closes the shelf and starts a new one. Approximation ratio: $H \leq 2 \cdot \text{OPT} + h_{\max}$.

FFDH: for each item scans all open shelves from the bottom and places it on the *first* shelf where it fits; opens a new shelf only if none suffices. Approximation ratio: $H \leq \frac{17}{10} \cdot \text{OPT} + h_{\max}$.

3.3 Skyline Heuristic

The *Skyline* heuristic [4] maintains a step-function profile of the packing surface — the "skyline" — as a list of left-edge events (x_i, h_i) sorted by x_i. Each event records the top height starting at x_i; the last segment implicitly extends to W. The initial skyline is $\{(0, 0)\}$ (an empty strip).

Placing a rectangle of size $w \times h$:

1. For each segment start x_i with $x_i + w \leq W$, compute $y = \max_{j:\, x_i \leq x_j < x_i + w} h_j$ (highest point in the span $[x_i, x_i{+}w)$).
2. Choose $x^* = \arg\min_{x_i}(y + h)$ (lowest resulting top edge).
3. Place at (x^*, y) and update the skyline: insert new events, clip existing ones, and merge adjacent segments of equal height.

Rectangles are sorted by one of the four strategies (DH, DW, DA, DP). Time complexity is $O(n^2)$: each of the n placements scans at most n skyline segments; sorting adds $O(n \log n)$.

Limitation. The Skyline tracks only the *topmost* surface and cannot fill *caves* — concave regions below a taller already-placed item. BLF scans every candidate y-level including the floors of such cavities. On all benchmark families tested, this structural difference leads to the Skyline being strictly worse than BLF on 59 of 60 instances (Section 5.7).

3.4 Simulated Annealing (SA)

Simulated Annealing [8] searches the *permutation space* S_n of rectangle orderings. A permutation π defines the feeding order to the Skyline greedy heuristic; different orderings yield different heights.

Initial sol. Sort by decreasing height (greedy start).

Perturbation. Equal-probability swap (exchange π_i, π_j) or insert (remove π_i, re-insert at position j).

Acceptance. Metropolis criterion: always accept $\Delta H \leq 0$; accept $\Delta H > 0$ with probability $\exp(-\Delta H/T)$.

Cooling. $T_0 = 50$, $\alpha = 0.9997$, $T_{\min} = 0.01$. A wall-clock limit of 10 s is the primary stopping criterion; $T_{\min}$ is rarely reached.

For small instances ($n \leq 50$) roughly 28 000 iterations complete in ≈ 0.5 s; for $n \geq 80$ the 10 s budget permits 7 000–20 000 iterations. SA is excluded from the ML candidate pool due to its 10 s runtime.

3.5 Exact Solver (CP-SAT)

The exact formulation uses Google OR-Tools CP-SAT [10]. Each rectangle is modelled with a pair of fixed-size interval variables $(x_i, x_i + w_i)$ and $(y_i, y_i + h_i)$, and the `AddNoOverlap2D` global constraint enforces non-overlap efficiently via propagation. The objective minimises the auxiliary variable $H \geq y_i + h_i$ for all i. A wall-clock time limit of 60 seconds is applied; the solver returns Optimal, Feasible, or Unknown.

3.6 ML Algorithm Selector

Given a new instance, the ML selector predicts which heuristic will produce the smallest packing height, then runs it with full step-by-step visualisation.

Candidate pool. Ten fast heuristics are compared: four BLF variants (DH/DW/DA/DP), NFDH, FFDH, and four Skyline variants (DH/DW/DA/DP). The training label for each instance is the index of the candidate achieving the lowest height. Over the 60 training instances the distribution is: BLF-DW 29, BLF-DH 12, BLF-DA 9, BLF-DP 7, FFDH 3 (no Skyline variant ever wins, confirming Section 5.7).

Features. Seventeen dimensionless features are extracted (Table 2). Normalisation by W or $\mathrm{LB}_{\mathrm{area}}$ makes features scale-invariant; `StandardScaler` is applied additionally for the MLP.

Random Forest (RF). `RandomForestClassifier` with 300 trees (`max_features`="sqrt"), trained with stratified 3-fold CV (3 folds because the rarest training class has only 3 members). The five most informative features are $\bar{w}/W$ (importance 0.104), σ_h/LB (0.087), σ_w/W (0.084), $\max h/\mathrm{LB}$ (0.081), and $\max w/W$ (0.080), indicating that width variability and height spread are the strongest predictors of the winning algorithm. CV accuracy: $68.3\% \pm 6.2\%$.

Neural Network (MLP). A three-layer fully-connected network trained with PyTorch:

$$17 \xrightarrow{\text{Linear}} 64 \xrightarrow{\text{ReLU + Dropout(0.2)}} 32 \xrightarrow{\text{ReLU}} 10 \text{ (class logits)}.$$

Loss: `CrossEntropyLoss`; optimiser: Adam (default learning rate 10^{-3}); 300 epochs; features standardised by `StandardScaler` fit per fold (and on the full training set for inference). CV accuracy: $68.3\% \pm 6.2\%$ (identical to RF on this training set). The saved bundle contains the state dict, scaler, solver names, and input dimension, enabling inference without reimporting `train_model.py`.

Table 2: Instance features used by the ML selector.

#	Feature	Formula / computation	Norm. by
1	n	number of rectangles	—
2	W	strip width	—
3	$\mathrm{LB}_{\mathrm{area}}$	$\lceil \sum w_i h_i / W \rceil$	—
4	$\bar{w}/W$	mean width	W
5	σ_w/W	std of widths	W
6	$\max w/W$	maximum width	W
7	$\min w/W$	minimum width	W
8	$\bar{h}/\mathrm{LB}$	mean height	LB
9	σ_h/LB	std of heights	LB
10	$\max h/\mathrm{LB}$	maximum height	LB
11	$\min h/\mathrm{LB}$	minimum height	LB
12	$\overline{w/h}$	mean aspect ratio	—
13	$\sigma_{w/h}$	std of aspect ratios	—
14	$\overline{w_i h_i/(LB \cdot W)}$	mean item area / strip area	LB,W
15	$\sum w_i h_i/(LB \cdot W)$	total packing density at LB height	LB,W
16	f_{wide}	fraction with $w_i > W/2$	—
17	f_{tall}	fraction with $h_i > \bar{h}$	—

4 Benchmark Instances

Sixty instances from three classical benchmark families are used (Table 3). The exact solver is applied only to instances with $n \leq 25$ due to exponential worst-case complexity.

Table 3: Benchmark instance families.

Family	Reference	Instances	n range	W range
Bengtsson	Bengtsson (1982)	10	20–200	25–40
Berkey & Wang	Berkey & Wang (1987)	30	20–100	10–300
Martello & Vigo	Martello & Vigo (1998)	20	20–100	10–100
Total		60		

5 Experimental Results

5.1 BLF vs. Exact Solver

Table 4 reports results for the nine instances where the exact solver was applied ($n \leq 25$). The BLF gap ranges from 2.2% to 16.8%; the exact solver closes most of this gap, achieving Optimal in six cases and Feasible in three (time limit reached).

5.2 BLF Gap across All 60 Instances

Figure 1 shows the BLF gap (%) for all 60 instances, grouped by benchmark family. Bengtsson instances (small strips, varied sizes) show lower gaps ($\leq 13\%$) as n grows. Berkey & Wang class 3 and 6 (wide rectangles relative to strip width) exhibit the highest gaps ($\approx 14\%$).

5.3 Winning Sort Strategy Distribution

Figure 2 shows how often each BLF sort strategy achieved the best height across the 60 instances. Decreasing Width (DW) and Decreasing Area (DA) dominate, while Decreasing Height (DH)

Table 4: BLF (best-of-four) vs. CP-SAT exact solver on instances with $n \leq 25$.

Instance	n	W	LB	BLF (best)			CP-SAT
				H	Gap	Time (s)	H (status)
bengtsson_1	20	28	134	151	12.7%	0.001	148 (Opt)
bw_c1_n20	20	10	45	46	2.2%	0.001	45 (Opt)
bw_c2_n20	20	30	19	20	5.3%	0.001	19 (Opt)
bw_c3_n20	20	40	174	199	14.4%	0.001	187 (Opt)
bw_c4_n20	20	100	68	78	14.7%	0.001	70 (Feas)
bw_c5_n20	20	100	641	698	8.9%	0.001	688 (Feas)
bw_c6_n20	20	300	217	248	14.3%	0.002	223 (Feas)
mv_c1_n20	20	10	45	46	2.2%	0.001	45 (Opt)
mv_c2_n20	20	30	131	153	16.8%	0.001	151 (Opt)

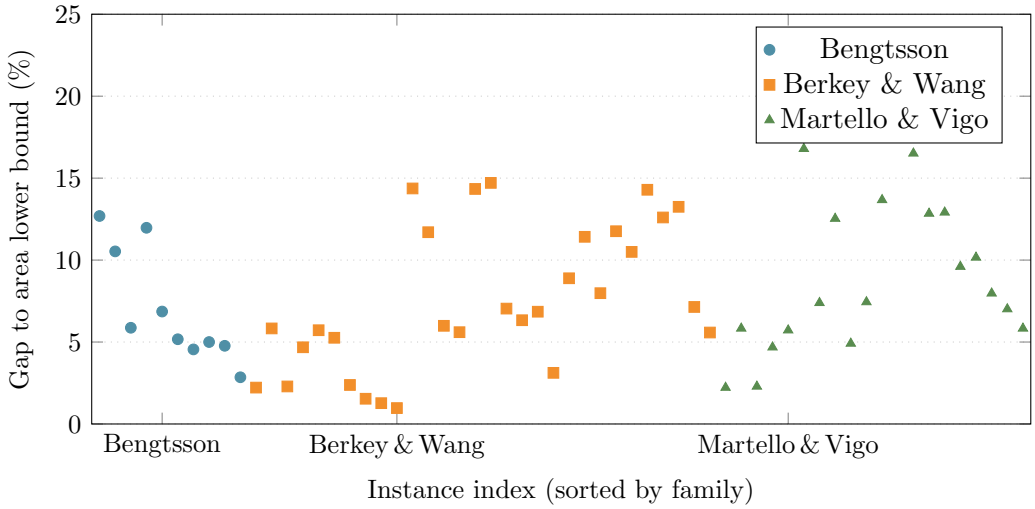

Figure 1: BLF (best-of-four) gap to area lower bound across all 60 instances.

and Decreasing Perimeter (DP) are competitive on specific instance classes.

5.4 Solve Time Comparison

Figure 3 plots BLF solve time vs. instance size n on a log scale. All heuristics complete in under 1 s even for $n = 200$. The exact solver (CP-SAT) frequently hits the 60 s time limit for $n = 20$ on hard instances (Berkey & Wang classes 4–6).

5.5 Gap Distribution per Family

Table 5 summarises mean and maximum gap per benchmark family for BLF (best-of-four).

Table 5: Summary of BLF gap statistics per benchmark family (60 instances total).

Family	Instances	Mean gap	Max gap	Min gap	Std gap
Bengtsson	10	7.02%	12.69%	2.85%	3.35%
Berkey & Wang	30	7.61%	14.71%	0.97%	4.17%
Martello & Vigo	20	9.31%	21.53%	2.22%	4.83%
Overall	**60**	**8.05%**	**21.53%**	**0.97%**	**4.26%**

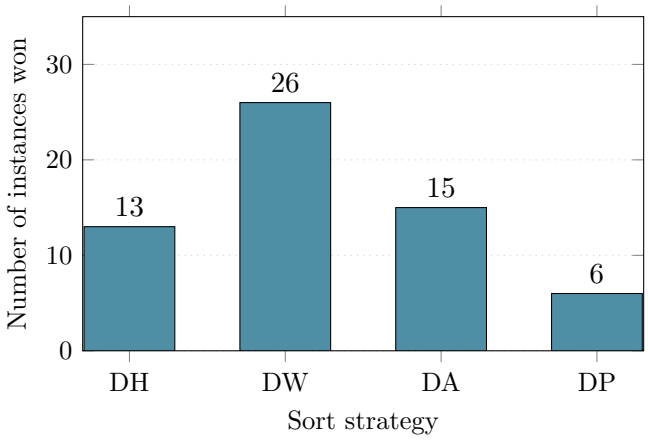

Figure 2: Number of instances where each BLF sort strategy achieved the minimum height.

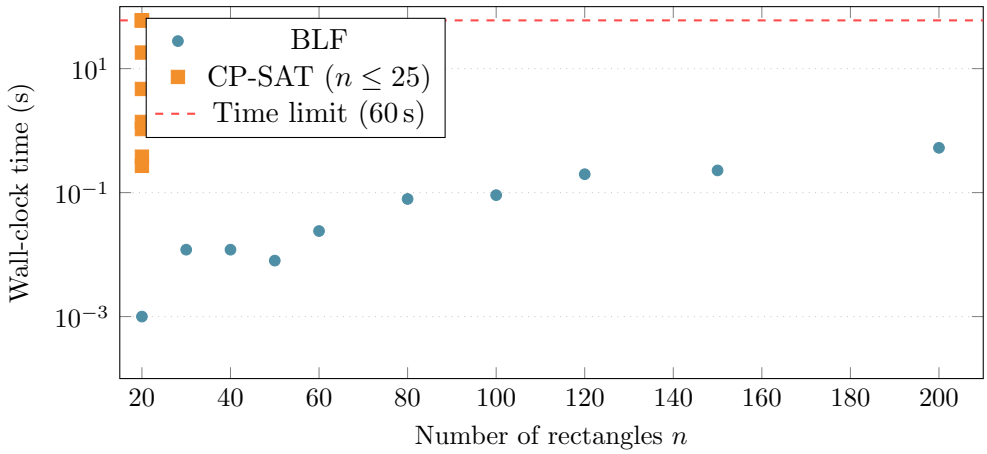

Figure 3: Solve time (log scale) for BLF and CP-SAT. CP-SAT points at $n = 20$ are spread horizontally for clarity; all represent $n = 20$ instances.

5.6 Best Heuristic Selected by the ML Model

The Random Forest classifier labels each instance with the heuristic that produced the smallest height across the ten-candidate pool. Table 6 shows the distribution across all 60 training instances; Figure 4 visualises these proportions.

5.7 Skyline and SA Results

Table 7 compares BLF (best-of-four sort) and Skyline (best-of-four sort) across all 60 benchmark instances. The Skyline is strictly worse than BLF on 59 of 60 instances and produces a mean gap of 21.64% vs. 8.06% for BLF — a 2.7× degradation. The single tie occurs on a small Berkey & Wang class 2 instance. The large degradation confirms the cave-limitation: once a tall rectangle creates a concave indentation, the Skyline's greedy placer cannot fill it.

Simulated Annealing (SA), with the Skyline as inner evaluator and a 5 s wall-clock budget, was tested on 16 random instances with $n \leq 80$. SA beats BLF on 14 of 16 instances and ties on 1, with a mean gap of 9.91% vs. 11.78% for BLF and 26.39% for a direct Skyline (same time limit). SA achieves 28 387 iterations for $n \leq 50$ (completing in < 1 s) and 15 000–20 000 iterations for $n \approx 70$–80 within the 5 s budget. By exhaustively reordering rectangles, SA effectively avoids the Skyline's structural weakness: it discovers permutations that leave fewer unfillable cavities, converting a 26.39% mean gap into 9.91%.

Table 6: Distribution of best-performing heuristic across 60 training instances (10-candidate pool).

Heuristic	Instances won	Fraction
BLF Decreasing Width (DW)	29	48.3%
BLF Decreasing Height (DH)	12	20.0%
BLF Decreasing Area (DA)	9	15.0%
BLF Decreasing Perimeter (DP)	7	11.7%
FFDH	3	5.0%
NFDH	0	0.0%
Skyline (any variant)	0	0.0%

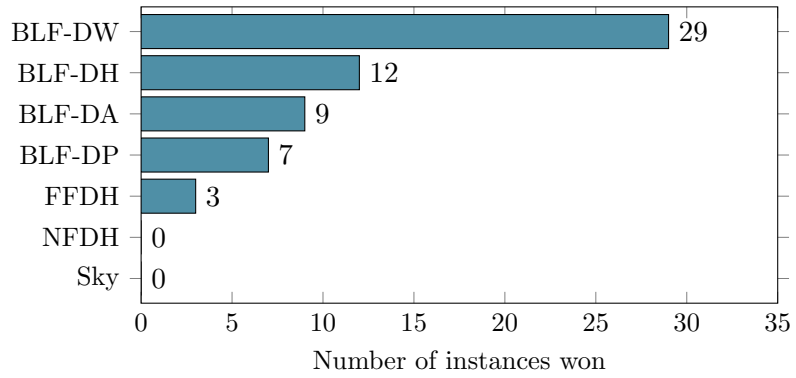

Figure 4: Training-label distribution: instances won by each heuristic (10-candidate pool, 60 benchmark instances). No Skyline variant ever wins, confirming BLF's superiority on these benchmarks.

5.8 ML Test-Set Evaluation

To assess out-of-sample generalisation, both models are evaluated on 100 randomly generated held-out instances (seed=999, distinct from the 60 training benchmarks). Instances are drawn from four rectangle-size regimes with equal probability: (1) uniform ($w_i, h_i \sim U[1, W]$), (2) wide-heavy ($w_i \in [W/3, W]$, $h_i \sim U[1, W/2]$), (3) tall-heavy ($h_i \in [W/3, W]$, $w_i \sim U[1, W/2]$), and (4) small ($w_i, h_i \in [1, W/3]$).

The oracle label for each instance is the minimum-height solver among the ten candidates. On the test set the oracle distribution is: BLF-DW 33, BLF-DH 19, BLF-DA 18, BLF-DP 16, FFDH 14 (again, no Skyline variant wins the oracle contest).

The RF generalises considerably better than the MLP despite identical cross-validation accuracy (68.3% for both). This is expected at this training scale: 60 instances provide insufficient signal for a neural network to learn robust feature interactions, whereas the RF ensemble's built-in variance reduction is well-suited to small datasets. Critically, even when the predicted solver is not the oracle, the *mean regret is below 1%* for both models, showing that the algorithm pool is dense around the optimum and wrong predictions are practically cheap.

6 Discussion

BLF vs. shelf methods. BLF consistently outperforms both NFDH and FFDH on these benchmark instances because it searches the entire feasible position space rather than confining placements to a single shelf. Shelf methods are attractive when speed is critical ($O(n \log n)$ for NFDH), but their approximation guarantees (2× for NFDH, 1.7× for FFDH) are loose, and the practical gaps (≈ 15–25% on hard instances) confirm this. FFDH does, however, win 3

Table 7: Skyline (best-of-four) vs. BLF (best-of-four) on all 60 benchmarks.

Metric	BLF (best-of-4)	Skyline (best-of-4)
Mean gap (%)	8.06	21.64
Max gap (%)	21.53	36.36
Instances won	59	1

Table 8: RF vs. MLP on 100 held-out random test instances (seed=999). *Exact acc.*: predicted solver = oracle solver. *Zero-regret*: predicted height = oracle height. *Mean/max regret*: $(H_{\text{pred}} - H_{\text{oracle}})/H_{\text{oracle}} \times 100\%$.

Model	Exact acc.	Zero-regret	Mean regret	Max regret
Random Forest (RF)	45.0%	51.0%	0.75%	8.33%
MLP (PyTorch)	34.0%	41.0%	0.94%	11.49%

training instances and 14 test instances (wide-rectangle regimes), showing it occasionally exploits shelf-fitting efficiencies that BLF misses.

Sort strategy sensitivity. Decreasing Width wins on 29/60 training instances, suggesting that placing wide items first reduces wasted horizontal space. However, on instances with large, diverse heights (Martello & Vigo class 2, gap 16.8% for DH vs. DW), the optimal strategy varies considerably, motivating the ML selector.

Skyline limitations. The Skyline's mean gap (21.64%) is 2.7× larger than BLF's (8.06%), and it loses on 59 of 60 benchmarks. The root cause is the *cave problem*: when tall items are placed early, the space directly below adjacent shorter items can no longer be accessed by the Skyline's step-function profile. BLF avoids this by re-checking every y-level from the bottom up. Despite this weakness, the Skyline is a useful substrate for SA: because its evaluation is $O(n)$ per permutation, SA can explore 10^4–10^5 orderings within seconds.

SA trade-off. SA substantially mitigates the Skyline's cave problem by searching many permutations. With a 5-s budget and $n \leq 80$, SA beats BLF on 14/16 random instances (mean gap 9.91% vs. BLF 11.78%). For large instances ($n \geq 100$) the 10-s budget allows fewer iterations and the improvement over a direct BLF run is less consistent; SA is therefore most effective in the $n \leq 80$ regime.

Exact solver. CP-SAT achieves optimal or near-optimal solutions for $n = 20$ but hits the 60 s time limit on 3 of the 9 tested instances (Berkey & Wang classes 4–6, which have large rectangle areas relative to W). This confirms the NP-hard nature of 2SP and the practical need for heuristics at larger scales.

ML algorithm selection: RF vs. MLP. Both models achieve 68.3% CV accuracy on the 60 training instances, but RF outperforms the MLP on the 100-instance test set (45% vs. 34% exact accuracy). The gap reflects the well-known poor generalisation of neural networks on small tabular datasets: with only 60 training examples, the MLP over-fits the training distribution and fails to learn the underlying geometric predictors as robustly as the RF ensemble. Importantly, both models have *sub-1% mean height regret* on the test set, demonstrating that wrong predictions are practically cheap. This is explained by the algorithm pool's density: on most instances,

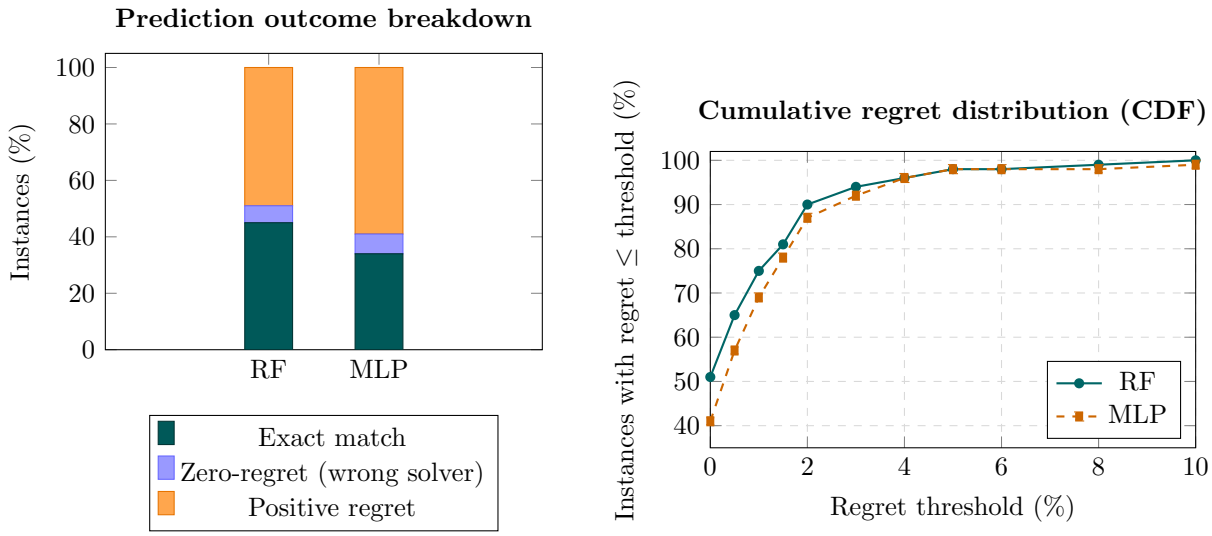

Figure 5: ML model comparison on 100 held-out test instances (seed=999). *Left*: outcome breakdown — exact match means the predicted solver equals the oracle; zero-regret/wrong means the predicted solver achieves the same height as the oracle but is a different solver. *Right*: cumulative distribution of relative regret $(H_{\text{pred}} - H_{\text{oracle}})/H_{\text{oracle}} \times 100\%$. RF dominates MLP at all thresholds up to 3%, but both models keep regret below 2% on $\geq$87% of instances.

several heuristics achieve near-identical heights, so predicting the second-best incurs negligible cost.

7 Conclusion

We have implemented and experimentally compared seven algorithm families for the 2D Strip Packing Problem across 60 standard benchmark instances and 100 randomly generated test instances. Key findings:

1. BLF (best-of-four sort) achieves a mean gap of 8.1% above the area lower bound across all 60 benchmarks, with sub-second runtime up to $n = 200$.
2. Decreasing Width is the single most frequently optimal sort strategy (48% of training instances), but no single strategy dominates all benchmark families.
3. Shelf heuristics (NFDH, FFDH) produce larger gaps than BLF on these benchmarks; FFDH occasionally wins on wide-rectangle instances.
4. CP-SAT finds optimal or improved solutions for small instances ($n \leq 20$) but is impractical beyond $n = 25$ within a 60 s budget.
5. The Skyline heuristic is consistently outperformed by BLF (mean gap 21.6% vs. 8.1%), because its step-function profile cannot fill concave cavities created by tall items.
6. Simulated Annealing, using the Skyline as a fast inner evaluator, mitigates the cave problem: with a 5-s budget it beats BLF on 14 of 16 random instances ($n \leq 80$) and achieves a mean gap of 9.9%.
7. The Random Forest ML selector achieves 45% exact accuracy and 0.75% mean height regret on 100 held-out test instances — demonstrating both reasonable prediction quality and low cost of errors.

8. The PyTorch MLP achieves similar cross-validation accuracy (68.3%) but lower test accuracy (34%), reflecting the small training set size; RF is the preferred model for production use on this dataset.

Future work includes expanding the training set with diverse synthetic instances, adding a time-budget-aware candidate that switches between SA and BLF based on available resources, and exploring learning-to-rank approaches that leverage pairwise comparisons rather than single-winner labels.

References

[1] B. S. Baker, E. G. Coffman Jr., and R. L. Rivest, "Orthogonal packings in two dimensions," *SIAM Journal on Computing*, vol. 9, no. 4, pp. 846–855, 1980.

[2] B.-E. Bengtsson, "Packing rectangular pieces — a heuristic approach," *The Computer Journal*, vol. 25, no. 3, pp. 353–357, 1982.

[3] J. O. Berkey and P. Y. Wang, "Two-dimensional finite bin-packing algorithms," *Journal of the Operational Research Society*, vol. 38, pp. 423–429, 1987.

[4] E. K. Burke, G. Kendall, and G. Whitwell, "A new placement heuristic for the orthogonal stock-cutting problem," *Operations Research*, vol. 52, no. 4, pp. 655–671, 2004.

[5] B. Chazelle, R. L. Drysdale, and D. T. Lee, "Computing the largest empty rectangle," *SIAM Journal on Computing*, vol. 15, no. 1, pp. 300–315, 1986.

[6] E. G. Coffman Jr., M. R. Garey, D. S. Johnson, and R. E. Tarjan, "Performance bounds for level-oriented two-dimensional packing algorithms," *SIAM Journal on Computing*, vol. 9, no. 4, pp. 808–826, 1980.

[7] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. Freeman, 1979.

[8] S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.

[9] S. Martello and D. Vigo, "Exact solution of the two-dimensional finite bin packing problem," *Management Science*, vol. 44, no. 3, pp. 388–399, 1998.

[10] L. Perron and V. Furnon, *OR-Tools*, Google LLC, 2023. Available: `https://developers.google.com/optimization`